SMARTSUPPORT AI

Task Assignment & Team Roles

Agentic AI-Powered E-Commerce Customer Support Platform

1. Work Division Principles

- Each member owns one clearly defined module, including its backend logic, related UI screens, and tests.
- Members may integrate with another module through agreed API contracts, but should not duplicate its business logic.
- The team agrees on shared data models, endpoint names, response formats, and branch rules before parallel implementation.
- Everyone contributes to integration, code review, bug fixing, and the final demo; these shared duties do not replace module ownership.

2. Member Responsibilities

Member 1 — Salma Mohamed Hafez Mahmoud

Primary role: Core E-commerce Operations & Order Management

- Implement customer/order data models and the order-related database tables using EF Core.
- Build secure backend operations for viewing order status and permitted customer/order details.
- Implement refund-request initiation and account-update business operations, including validation and business rules.
- Create the related customer/order pages or components in MVC, using the agreed API/service contracts.
- Write tests for order lookup, invalid requests, ownership checks, and refund-request rules.

Main deliverable: Working order lookup and customer business-operation module, with validation and tests.

Member 2 — Engy Abdelrahman Mohamed Nagy

Primary role: Agentic AI, LLM Integration & Knowledge Retrieval

- Integrate the selected LLM API behind a provider abstraction so it can be replaced if needed.
- Implement AI orchestration with Microsoft Semantic Kernel, prompt design, intent handling, and conversation context.
- Build the knowledge base retrieval flow for FAQs, shipping policies, return policies, and support articles (RAG).
- Define AI tool-calling contracts and connect the agent to approved backend tools without giving it direct database access.
- Implement safe fallback responses and tests for retrieval quality, unsupported requests, and tool-call selection.

Main deliverable: AI service that answers policy questions using approved sources and requests authorized backend actions through defined tools.

Member 3 — Ali Mohamed Abdelmawgoud Nassef

Primary role: Authentication, Authorization & Conversation API

- Configure ASP.NET Identity, login/logout, user registration if required, and role-based authorization.
- Define roles and access rules for Customer, Support Agent, Support Manager, and Administrator.
- Implement the core conversation/session API and persistence of messages needed to maintain chat context.
- Apply server-side validation, authorization checks, secure configuration practices, and consistent API error responses.
- Write tests for authentication, role permissions, unauthorized access, and conversation access boundaries.

Main deliverable: Secure identity and conversation foundation that other modules can use consistently.

Member 4 — Khaled Osama Abdellatef Selem

Primary role: Support Tickets, Escalation & Real-Time Agent Collaboration

- Implement ticket creation, assignment, priority, status changes, and ticket history.
- Build escalation rules for complex, sensitive, or unresolved conversations.
- Implement the human-agent workflow, including access to conversation history, customer context, AI summary, and escalation reason.
- Use SignalR for real-time customer-agent messaging or conversation updates, coordinating with the conversation API owner.
- Build the related support-agent screens and tests for ticket lifecycle, escalation, and real-time events.

Main deliverable: End-to-end ticket and escalation workflow with agent collaboration and real-time updates.

Member 5 — Zeyad Mohamed Samir Mohamed

Primary role: Administration, Knowledge Management, Analytics & Integration

- Build administration functions for managing users/roles where authorized, knowledge articles, and configurable support settings.
- Implement support analytics such as ticket volume, response time, resolution status, and escalation patterns.
- Implement audit-log viewing and basic operational monitoring views, coordinating event formats with other module owners.
- Build the related manager/admin dashboard pages and connect them to the agreed backend endpoints.
- Own the integration checklist and Docker Compose/development setup, coordinate integration builds, and test cross-module flows.

Main deliverable: Admin/manager dashboard, knowledge-management screens, basic analytics, and a repeatable integrated development setup.

3. Shared Responsibilities (All Members)

- Attend short weekly planning and integration meetings; raise blockers early.
- Use Git feature branches and pull requests; review at least one teammate's change when practical.
- Document module setup, endpoints, required environment variables, and known limitations.
- Contribute to end-to-end testing, bug fixing, and the final presentation/demo.
- Never commit API keys, passwords, connection-string secrets, or real customer data to GitHub.

4. Integration Rules — To Prevent Confusion

1. **Agree on the architecture first:** Use a layered structure (Presentation, Business/Application, Data Access, AI/Integration). Keep controllers thin and business rules in services.
2. **One owner per business rule:** Order/refund rules belong to Member 1; identity/permissions to Member 3; ticket/escalation rules to Member 4; AI orchestration/RAG to Member 2; admin reporting and integration coordination to Member 5.
3. **Define contracts before coding:** Agree on DTOs, endpoint routes, status codes, error format, and ownership of shared entities before implementation. Publish these in the repository README or an API document.
4. **Avoid editing the same files:** Each member works mainly in their module folders. If a shared file must change, raise it in the team chat and coordinate before merging.
5. **Use mock interfaces early:** If a dependency is not ready, use a small mock service or agreed interface instead of waiting or independently rebuilding the same feature.
6. **Protect main:** Use feature branches and pull requests. Merge only after the module builds and basic tests pass; integrate small changes frequently.
7. **Keep scope realistic:** Prioritize the core demo: customer chat, policy-based answer, order-status lookup, refund request workflow, ticket escalation to a human, and admin/manager visibility. Treat optional deployment/cloud enhancements as stretch goals.

5. Suggested 6–8 Week Plan

Phase	Timing	Expected outcome
Planning & contracts	Week 1	Agree on database entities, module boundaries, API/DTO contracts, repo structure, and MVP scope.
Parallel module development	Weeks 2–4	Each member builds their owned module with unit tests and a small working UI/API slice.
Integration	Weeks 5–6	Connect AI tools to backend operations, chat to SignalR/tickets, and admin views to metrics/data.
System testing & polish	Week 7	Test end-to-end workflows, authorization, failure cases, and fix integration bugs.
Demo & documentation	Week 8 (if available)	Finalize README, setup instructions, demo data, presentation, and deployment preparation.

6. Definition of Done for Every Module

- The feature works against the agreed interface and does not duplicate another module's logic.
- Inputs are validated and permissions are enforced on the server.
- Core success and failure cases have tests or a documented manual test checklist.
- The code builds, is pushed to a feature branch, and has a clear README/setup note where needed.
- The module owner demonstrates the feature during integration and resolves bugs in their area.